

FILE WITH CODE FOR HOME WORK 3

```
# Read the dataset
```

```
cat("\n[1] Loading data...\n")
```

```
data <- read.csv("data_for_analysis.csv", stringsAsFactors = FALSE)
```

```
# Define hormone columns for analysis
```

```
hormone_columns <- c(
```

```
  "hormone1", "hormone2", "hormone3", "hormone4",
```

```
  "hormone5", "hormone6", "hormone7", "hormone8",
```

```
  "hormone10_generated"
```

```
)
```

```
# Remove rows with missing outcome or hormone values
```

```
valid_rows <- complete.cases(data[, c("outcome", hormone_columns)])
```

```
hormone_data <- data[valid_rows, ]
```

```
# Split data by outcome group
```

```
group_0 <- hormone_data[hormone_data$outcome == 0, hormone_columns]
```

```
group_1 <- hormone_data[hormone_data$outcome == 1, hormone_columns]
```

```
cat(" Total observations:", nrow(hormone_data), "\n")
```

```
cat(" Group 0 (Outcome=0):", nrow(group_0), "samples\n")
```

```
cat(" Group 1 (Outcome=1):", nrow(group_1), "samples\n")
```

```
# -----
```

```
# STEP 2: HELPER FUNCTIONS FOR STATISTICS
```

```
# -----
```

```
# Calculate skewness (measure of distribution asymmetry)
```

```
compute_skewness <- function(x) {
```

```
  x <- x[!is.na(x)]
```

```
  n <- length(x)
```

```
  if (n < 3) return(NA)
```

```
  m <- mean(x)
```

```
  s <- sd(x)
```

```
  if (s == 0) return(NA)
```

```
  return(sum(((x - m) / s)^3) * n / ((n - 1) * (n - 2)))
```

```
}
```

```
# Calculate kurtosis (measure of tail heaviness)
```

```
compute_kurtosis <- function(x) {
```

```
  x <- x[!is.na(x)]
```

```
  n <- length(x)
```

```
  if (n < 4) return(NA)
```

```

m <- mean(x)

s <- sd(x)

if (s == 0) return(NA)

kurt <- sum(((x - m) / s)^4) * n * (n + 1) / ((n - 1) * (n - 2) * (n - 3))

excess <- 3 * (n - 1)^2 / ((n - 2) * (n - 3))

return(kurt - excess)

}

# Levene's test for equality of variances (manual implementation)

levene_variance_test <- function(values, group_labels) {

  unique_groups <- unique(group_labels)

  group_medians <- sapply(unique_groups, function(g)
    median(values[group_labels == g]))

  deviations <- abs(values - group_medians[match(group_labels, unique_groups)])

  anova_result <- summary(aov(deviations ~ factor(group_labels)))

  return(anova_result[[1]][["Pr(>F)"]][1])

}

# Brunner-Munzel test (non-parametric, robust to unequal variances)

brunner_munzel_test <- function(x, y) {

  x <- x[!is.na(x)]

  y <- y[!is.na(y)]

  nx <- length(x)

```

```
ny <- length(y)
```

```
# Combine and rank all values
```

```
all_values <- c(x, y)
```

```
all_ranks <- rank(all_values)
```

```
ranks_x <- all_ranks[1:nx]
```

```
ranks_y <- all_ranks[(nx + 1):(nx + ny)]
```

```
# Calculate test statistic
```

```
mean_rank_x <- mean(ranks_x)
```

```
mean_rank_y <- mean(ranks_y)
```

```
var_x <- var(ranks_x)
```

```
var_y <- var(ranks_y)
```

```
t_stat <- (mean_rank_x - mean_rank_y) / sqrt((var_x / nx) + (var_y / ny))
```

```
# Degrees of freedom (Satterthwaite approximation)
```

```
df <- (var_x / nx + var_y / ny)^2 /
```

```
  ((var_x / nx)^2 / (nx - 1) + (var_y / ny)^2 / (ny - 1))
```

```
return(2 * pt(-abs(t_stat), df))
```

```
}
```

```
# -----  
# STEP 3: DESCRIPTIVE STATISTICS  
# -----
```

```
cat("\n[2] Calculating descriptive statistics...\n")
```

```
create_descriptive_table <- function(data, group_name) {  
  results <- data.frame(  
    Hormone = character(),  
    N = integer(),  
    Mean = numeric(),  
    SD = numeric(),  
    Median = numeric(),  
    Minimum = numeric(),  
    Maximum = numeric(),  
    Skewness = numeric(),  
    Kurtosis = numeric(),  
    Group = character(),  
    stringsAsFactors = FALSE  
  )
```

```
  for (hormone in colnames(data)) {  
    values <- data[[hormone]]
```

```

values <- values[!is.na(values)]

results <- rbind(results, data.frame(
  Hormone = hormone,
  N = length(values),
  Mean = round(mean(values), 4),
  SD = round(sd(values), 4),
  Median = round(median(values), 4),
  Minimum = round(min(values), 4),
  Maximum = round(max(values), 4),
  Skewness = round(compute_skewness(values), 4),
  Kurtosis = round(compute_kurtosis(values), 4),
  Group = group_name,
  stringsAsFactors = FALSE
))
}
return(results)
}

descriptive_group0 <- create_descriptive_table(group_0, "Outcome = 0")
descriptive_group1 <- create_descriptive_table(group_1, "Outcome = 1")
descriptive_all <- rbind(descriptive_group0, descriptive_group1)

```

```
cat("\n--- Descriptive Statistics Summary ---\n")
```

```
print(descriptive_all)
```

```
# -----
```

```
# STEP 4: NORMALITY AND HOMOGENEITY TESTS
```

```
# -----
```

```
cat("\n[3] Performing normality and homogeneity tests...\n")
```

```
normality_results <- data.frame(
```

```
  Hormone = character(),
```

```
  Shapiro_Group0_P = numeric(),
```

```
  Shapiro_Group1_P = numeric(),
```

```
  Normal_Group0 = character(),
```

```
  Normal_Group1 = character(),
```

```
  Levene_P = numeric(),
```

```
  Equal_Variance = character(),
```

```
  stringsAsFactors = FALSE
```

```
)
```

```
for (hormone in hormone_columns) {
```

```
  # Extract values for each group
```

```
  vals_0 <- group_0[[hormone]][!is.na(group_0[[hormone]])]
```

```

vals_1 <- group_1[[hormone]][!is.na(group_1[[hormone]])]

# Shapiro-Wilk tests
shapiro_0 <- tryCatch(shapiro.test(vals_0)$p.value, error = function(e) NA)
shapiro_1 <- tryCatch(shapiro.test(vals_1)$p.value, error = function(e) NA)

# Levene's test
all_vals <- c(vals_0, vals_1)
all_groups <- c(rep(0, length(vals_0)), rep(1, length(vals_1)))
levene_p <- tryCatch(levene_variance_test(all_vals, all_groups), error =
function(e) NA)

normality_results <- rbind(normality_results, data.frame(
  Hormone = hormone,
  Shapiro_Group0_P = round(shapiro_0, 4),
  Shapiro_Group1_P = round(shapiro_1, 4),
  Normal_Group0 = ifelse(!is.na(shapiro_0) && shapiro_0 > 0.05, "Yes", "No"),
  Normal_Group1 = ifelse(!is.na(shapiro_1) && shapiro_1 > 0.05, "Yes", "No"),
  Levene_P = round(levene_p, 4),
  Equal_Variance = ifelse(!is.na(levene_p) && levene_p > 0.05, "Yes", "No"),
  stringsAsFactors = FALSE
))
}

```

```

cat("\n--- Normality and Homogeneity Test Results ---\n")
print(normality_results)

# -----

# STEP 5: VISUALIZATIONS (HISTOGRAMS AND Q-Q PLOTS)
# -----

cat("\n[4] Generating visualizations...\n")

# Create output directory
dir.create("hormone_plots", showWarnings = FALSE)

# Function to create histogram
draw_histogram <- function(values, hormone_name, group_label) {
  values <- values[!is.na(values)]
  if (length(values) < 3) return()

  hist(values,
        main = paste(hormone_name, "-", group_label),
        xlab = "Hormone Level",
        ylab = "Frequency",
        col = "steelblue",

```

```
border = "black",  
breaks = "Sturges")
```

```
density_curve <- density(values, na.rm = TRUE)  
lines(density_curve, col = "red", lwd = 2)  
}
```

```
# Function to create Q-Q plot
```

```
draw_qqplot <- function(values, hormone_name, group_label) {  
  values <- values[!is.na(values)]  
  if (length(values) < 3) return()
```

```
  n <- length(values)  
  theoretical_q <- qnorm((1:n - 0.5) / n)  
  sample_q <- sort(values)
```

```
  plot(theoretical_q, sample_q,  
        main = paste(hormone_name, "-", group_label),  
        xlab = "Theoretical Quantiles",  
        ylab = "Sample Quantiles",  
        pch = 16, col = "blue", cex = 0.8)
```

```
# Add reference line through quartiles
```

```

quartiles <- quantile(values, c(0.25, 0.75), na.rm = TRUE)
theo_quartiles <- qnorm(c(0.25, 0.75))
slope <- diff(quartiles) / diff(theo_quartiles)
intercept <- quartiles[1] - slope * theo_quartiles[1]
abline(a = intercept, b = slope, col = "red", lwd = 2)
}

# Generate individual plots for each hormone
for (hormone in hormone_columns) {
  values_0 <- group_0[[hormone]]
  values_1 <- group_1[[hormone]]

  # Histograms

  png(paste0("hormone_plots/", hormone, "_group0_histogram.png"), width = 600,
height = 500)

  draw_histogram(values_0, hormone, "Outcome = 0")

  dev.off()

  png(paste0("hormone_plots/", hormone, "_group1_histogram.png"), width = 600,
height = 500)

  draw_histogram(values_1, hormone, "Outcome = 1")

  dev.off()
}

```

```

# Q-Q plots

png(paste0("hormone_plots/", hormone, "_group0_qqplot.png"), width = 600,
height = 500)

draw_qqplot(values_0, hormone, "Outcome = 0")

dev.off()


png(paste0("hormone_plots/", hormone, "_group1_qqplot.png"), width = 600,
height = 500)

draw_qqplot(values_1, hormone, "Outcome = 1")

dev.off()

}


# Create combined PDF with all plots

pdf("hormone_plots/all_hormone_plots_combined.pdf", width = 12, height = 8)


for (hormone in hormone_columns) {
  values_0 <- group_0[[hormone]]
  values_1 <- group_1[[hormone]]


  par(mfrow = c(2, 2))


  draw_histogram(values_0, hormone, "Outcome = 0")
  draw_histogram(values_1, hormone, "Outcome = 1")

```

```

draw_qqplot(values_0, hormone, "Outcome = 0")
draw_qqplot(values_1, hormone, "Outcome = 1")

mtext(paste("Distribution of", hormone), side = 3, line = -2, outer = TRUE, cex =
1.5)
}

dev.off()

cat(" Visualizations saved to 'hormone_plots' directory\n")

# -----
# STEP 6: STATISTICAL GROUP COMPARISONS
# -----

cat("\n[5] Performing statistical group comparisons...\n")

comparison_results <- data.frame(
  Hormone = character(),
  T_Test_P = numeric(),
  Welch_Test_P = numeric(),
  Wilcoxon_P = numeric(),
  Brunner_Munzel_P = numeric(),

```

```

Recommended_Test = character(),
Significant = character(),
stringsAsFactors = FALSE
)

for (hormone in hormone_columns) {
  vals_0 <- group_0[[hormone]][!is.na(group_0[[hormone]])]
  vals_1 <- group_1[[hormone]][!is.na(group_1[[hormone]])]

  if (length(vals_0) < 3 | length(vals_1) < 3) {
    comparison_results <- rbind(comparison_results, data.frame(
      Hormone = hormone,
      T_Test_P = NA, Welch_Test_P = NA, Wilcoxon_P = NA, Brunner_Munzel_P
= NA,
      Recommended_Test = "Insufficient data", Significant = "Cannot determine",
      stringsAsFactors = FALSE
    ))
    next
  }

  # Perform various tests

  t_test_p <- tryCatch(t.test(vals_0, vals_1, var.equal = TRUE)$p.value, error =
function(e) NA)

```

```
welch_p <- tryCatch(t.test(vals_0, vals_1, var.equal = FALSE)$p.value, error =  
function(e) NA)
```

```
wilcox_p <- tryCatch(wilcox.test(vals_0, vals_1)$p.value, error = function(e) NA)
```

```
bm_p <- tryCatch(brunner_munzel_test(vals_0, vals_1), error = function(e) NA)
```

```
# Get test recommendations based on earlier results
```

```
norm0 <- normality_results[normality_results$Hormone == hormone,  
"Normal_Group0"]
```

```
norm1 <- normality_results[normality_results$Hormone == hormone,  
"Normal_Group1"]
```

```
eq_var <- normality_results[normality_results$Hormone == hormone,  
"Equal_Variance"]
```

```
if (norm0 == "Yes" && norm1 == "Yes" && eq_var == "Yes"  
&& !is.na(t_test_p)) {
```

```
  recommended <- "T-test (parametric, equal variances)"
```

```
} else if (norm0 == "Yes" && norm1 == "Yes" && eq_var == "No"  
&& !is.na(welch_p)) {
```

```
  recommended <- "Welch's T-test (parametric, unequal variances)"
```

```
} else {
```

```
  recommended <- "Brunner-Munzel (non-parametric, robust)"
```

```
}
```

```
significant <- ifelse(!is.na(bm_p) && bm_p < 0.05, "Yes (p < 0.05)", "No")
```

```

comparison_results <- rbind(comparison_results, data.frame(
  Hormone = hormone,
  T_Test_P = round(t_test_p, 4),
  Welch_Test_P = round(welch_p, 4),
  Wilcoxon_P = round(wilcox_p, 4),
  Brunner_Munzel_P = round(bm_p, 4),
  Recommended_Test = recommended,
  Significant = significant,
  stringsAsFactors = FALSE
))
}

cat("\n--- Statistical Test Results ---\n")
print(comparison_results)

# -----
# STEP 7: CORRELATION ANALYSIS WITH HEATMAPS
# -----

cat("\n[6] Performing correlation analysis...\n")

# Determine appropriate correlation method based on normality

```

```
normal_count_0 <- sum(normality_results$Normal_Group0 == "Yes", na.rm = TRUE)
```

```
normal_count_1 <- sum(normality_results$Normal_Group1 == "Yes", na.rm = TRUE)
```

```
total_hormones <- length(hormone_columns)
```

```
cor_method_0 <- ifelse(normal_count_0 > total_hormones / 2, "pearson",  
"spearman")
```

```
cor_method_1 <- ifelse(normal_count_1 > total_hormones / 2, "pearson",  
"spearman")
```

```
cat("\n Correlation method for Group 0:", toupper(cor_method_0), "\n")
```

```
cat(" Correlation method for Group 1:", toupper(cor_method_1), "\n")
```

```
# Function to compute correlation matrix
```

```
compute_correlation_matrix <- function(data, method) {
```

```
  n_vars <- ncol(data)
```

```
  cor_matrix <- matrix(1, nrow = n_vars, ncol = n_vars)
```

```
  colnames(cor_matrix) <- colnames(data)
```

```
  rownames(cor_matrix) <- colnames(data)
```

```
  for (i in 1:(n_vars - 1)) {
```

```
    for (j in (i + 1):n_vars) {
```

```
      complete_idx <- complete.cases(data[, c(i, j)])
```

```

x <- data[complete_idx, i]
y <- data[complete_idx, j]

if (length(x) > 2) {
  if (method == "pearson") {
    cor_matrix[i, j] <- cor(x, y, method = "pearson")
  } else {
    cor_matrix[i, j] <- cor(rank(x), rank(y), method = "pearson")
  }
  cor_matrix[j, i] <- cor_matrix[i, j]
}
}
}

return(cor_matrix)
}

# Compute correlation matrices

cor_matrix_0 <- compute_correlation_matrix(group_0, cor_method_0)
cor_matrix_1 <- compute_correlation_matrix(group_1, cor_method_1)

cat("\n--- Correlation Matrices ---\n")

cat("\nGroup 0 (Outcome = 0):\n")

print(round(cor_matrix_0, 3))

```

```
cat("\nGroup 1 (Outcome = 1):\n")
print(round(cor_matrix_1, 3))

# Create heatmap function
create_heatmap <- function(cor_matrix, title, filename) {
  png(filename, width = 800, height = 800)

  color_palette <- colorRampPalette(c("blue", "white", "red"))(100)

  heatmap(cor_matrix,
          col = color_palette,
          symm = TRUE,
          main = title,
          margins = c(10, 10),
          cexCol = 0.8,
          cexRow = 0.8,
          key = TRUE,
          key.title = "Correlation",
          trace = "none")

  dev.off()
}
```

```
# Generate heatmaps
```

```
create_heatmap(cor_matrix_0,  
               paste("Correlation Heatmap - Outcome = 0\nMethod:",  
toupเปอร์(cor_method_0)),  
               "hormone_plots/correlation_heatmap_group0.png")
```

```
create_heatmap(cor_matrix_1,  
               paste("Correlation Heatmap - Outcome = 1\nMethod:",  
toupเปอร์(cor_method_1)),  
               "hormone_plots/correlation_heatmap_group1.png")
```

```
cat(" Correlation heatmaps saved to 'hormone_plots' directory\n")
```

```
# -----
```

```
# STEP 8: EXPORT ALL RESULTS
```

```
# -----
```

```
cat("\n[7] Exporting results...\n")
```

```
# Add significance flags to test results
```

```
normality_results$Interpretation_Group0 <- ifelse(  
  normality_results$Shapiro_Group0_P < 0.05,
```

```

    "Non-normal distribution",
    "Normal distribution"
)

normality_results$Interpretation_Group1 <- ifelse(
  normality_results$Shapiro_Group1_P < 0.05,
  "Non-normal distribution",
  "Normal distribution"
)

normality_results$Variance_Interpretation <- ifelse(
  normality_results$Levene_P < 0.05,
  "Unequal variances",
  "Equal variances"
)

# Export to CSV files

write.csv(descriptive_all, "01_descriptive_statistics.csv", row.names = FALSE)
write.csv(normality_results, "02_normality_tests.csv", row.names = FALSE)
write.csv(comparison_results, "03_group_comparisons.csv", row.names = FALSE)

# Export correlation matrices

cor_matrix_0_df <- as.data.frame(cor_matrix_0)
cor_matrix_0_df$Variable <- rownames(cor_matrix_0)

```

```
cor_matrix_0_df <- cor_matrix_0_df[, c(ncol(cor_matrix_0_df),
1:(ncol(cor_matrix_0_df) - 1))]
```

```
write.csv(cor_matrix_0_df, "04_correlation_matrix_group0.csv", row.names =
FALSE)
```

```
cor_matrix_1_df <- as.data.frame(cor_matrix_1)
```

```
cor_matrix_1_df$Variable <- rownames(cor_matrix_1)
```

```
cor_matrix_1_df <- cor_matrix_1_df[, c(ncol(cor_matrix_1_df),
1:(ncol(cor_matrix_1_df) - 1))]
```

```
write.csv(cor_matrix_1_df, "05_correlation_matrix_group1.csv", row.names =
FALSE)
```

```
cat(" Results exported to CSV files:\n")
```

```
cat(" - 01_descriptive_statistics.csv\n")
```

```
cat(" - 02_normality_tests.csv\n")
```

```
cat(" - 03_group_comparisons.csv\n")
```

```
cat(" - 04_correlation_matrix_group0.csv\n")
```

```
cat(" - 05_correlation_matrix_group1.csv\n")
```

```
sink("06_complete_analysis_summary.txt")
```

```
cat("=====  
=====\n")
```

```
cat("COMPLETE HORMONE ANALYSIS SUMMARY REPORT\n")
```

```
cat("=====
=====\\n")
```

```
cat("Generated on:", date(), "\\n\\n")
```

```
cat("SECTION 1: DATA OVERVIEW\\n")
```

```
cat("-----\\n")
```

```
cat("Total observations analyzed:", nrow(hormone_data), "\\n")
```

```
cat("Group 0 (Outcome = 0) size:", nrow(group_0), "\\n")
```

```
cat("Group 1 (Outcome = 1) size:", nrow(group_1), "\\n")
```

```
cat("Number of hormones analyzed:", length(hormone_columns), "\\n\\n")
```

```
cat("SECTION 2: NORMALITY ASSESSMENT (Shapiro-Wilk Test,  $\alpha = 0.05$ )\\n")
```

```
cat("-----\\n")
```

```
cat("\\nHormones with normal distribution in Group 0:\\n")
```

```
normal_0_list <- normality_results$Hormone[normality_results$Normal_Group0
== "Yes"]
```

```
cat(if (length(normal_0_list) > 0) paste(normal_0_list, collapse = ", ") else
"None\\n")
```

```
cat("\\nHormones with normal distribution in Group 1:\\n")
```

```
normal_1_list <- normality_results$Hormone[normality_results$Normal_Group1
== "Yes"]
```

```
cat(if (length(normal_1_list) > 0) paste(normal_1_list, collapse = ", ") else
"None\\n\\n")
```

```
cat("SECTION 3: VARIANCE HOMOGENEITY (Levene's Test,  $\alpha = 0.05$ )\n")
```

```
cat("-----\n")
```

```
cat("\nHormones with equal variances between groups:\n")
```

```
equal_var_list <- normality_results$Hormone[normality_results$Equal_Variance  
== "Yes"]
```

```
cat(if (length(equal_var_list) > 0) paste(equal_var_list, collapse = ", ") else  
"None\n\n")
```

```
cat("SECTION 4: STATISTICAL TEST RECOMMENDATIONS\n")
```

```
cat("-----\n")
```

```
for (i in 1:nrow(comparison_results)) {
```

```
  cat("\n", comparison_results$Hormone[i], ":\n", sep = "")
```

```
  cat(" Recommended test:", comparison_results$Recommended_Test[i], "\n")
```

```
  cat(" Brunner-Munzel p-value:", comparison_results$Brunner_Munzel_P[i], "\n")
```

```
  cat(" Significant difference:", comparison_results$Significant[i], "\n")
```

```
}
```

```
cat("\nSECTION 5: SIGNIFICANT FINDINGS (Brunner-Munzel Test,  $p < 0.05$ )\n")
```

```
cat("-----\n")
```

```
significant_hormones
```

```
<-
```

```
comparison_results[comparison_results$Brunner_Munzel_P < 0.05, ]
```

```
if (nrow(significant_hormones) > 0) {
```

```

for (i in 1:nrow(significant_hormones)) {
  cat(" ", significant_hormones$Hormone[i], ": p = ",
      significant_hormones$Brunner_Munzel_P[i], "\n", sep = "")
}
} else {
  cat(" No hormones showed statistically significant differences\n")
}

cat("\nSECTION 6: CORRELATION ANALYSIS\n")
cat("-----\n")
cat("Correlation method for Group 0:", toupper(cor_method_0), "\n")
cat("Correlation method for Group 1:", toupper(cor_method_1), "\n")
cat("\nMethod selection rationale:\n")
cat(" - Pearson: Assumes normally distributed data\n")
cat(" - Spearman: Non-parametric, suitable for non-normal data\n")

# Find strongest correlations
cat("\nStrongest correlations in Group 0 ( $|r| > 0.6$ ):\n")
strong_cors_0 <- 0
for (i in 1:ncol(cor_matrix_0)) {
  for (j in 1:ncol(cor_matrix_0)) {
    if (i < j && abs(cor_matrix_0[i, j]) > 0.6) {
      cat(" ", colnames(cor_matrix_0)[i], " vs ", colnames(cor_matrix_0)[j],

```

```

      ": r = ", round(cor_matrix_0[i, j], 3), "\n", sep = "")
    strong_cors_0 <- strong_cors_0 + 1
  }
}
}
if (strong_cors_0 == 0) cat(" No strong correlations ( $|r| > 0.6$ ) found\n")

cat("\nStrongest correlations in Group 1 ( $|r| > 0.6$ ):\n")
strong_cors_1 <- 0
for (i in 1:ncol(cor_matrix_1)) {
  for (j in 1:ncol(cor_matrix_1)) {
    if (i < j && abs(cor_matrix_1[i, j]) > 0.6) {
      cat(" ", colnames(cor_matrix_1)[i], " vs ", colnames(cor_matrix_1)[j],
        ": r = ", round(cor_matrix_1[i, j], 3), "\n", sep = "")
      strong_cors_1 <- strong_cors_1 + 1
    }
  }
}
if (strong_cors_1 == 0) cat(" No strong correlations ( $|r| > 0.6$ ) found\n")

```